



Final Deliverable

Project Title: Cyber Network Threat Analyzer

Instructor: Sir Zubair Ahmed

Course: Data Structures and Algorithms (CS221)

NAME	REG: NO
Abdullah Munir	2024042
Hasnain Zia Wazir	2024222
Hadeed Haider	2024189

1. Overview of Phase 2 Implementation

In the previous phase, we established the core network using Linked Lists (Adjacency Lists) and simulated basic infection spread using a Queue (BFS). In this phase (Final Deliverable), we have expanded the project to include network analysis and threat prediction.

The key additions in this module are:

- Patient Zero Detection: Using DFS (Depth First Search) to trace the most likely source of an infection cluster.
 - Risk Profiling: Using Selection Sort to rank devices based on suspicious behaviour.
 - Heuristic Analysis: Mathematical formulas to predict future attacks based on node degree and infection history.
-

2. New Algorithms & Data Structures

A. Depth First Search (DFS) for Reachability

While BFS was used for spreading infection layer-by-layer, we implemented DFS in the NetworkAnalyzer class to determine the "impact scope" of a node.

- Purpose: To find "Patient Zero." By running a recursive DFS from infected nodes, we calculate how many *other* infected nodes can be reached from a specific device. The device that can reach the highest number of other infected nodes is identified as the likely source.
- Implementation: The function dfsReachability recursively visits connected nodes to count the cluster size.

B. Selection Sort for Risk Ranking

In the Simulator class, we implemented a sorting algorithm to organize the "Behaviour Scores."

- Purpose: To display a leaderboard of the most dangerous or vulnerable devices (High/Medium/Low risk).
- Implementation: We used Selection Sort. Since the maximum number of devices is capped at 200 (MAX_DEVICES), Selection Sort provides a stable and easy-to-implement solution for ranking the behaviorPercent array in descending order.

C. Heuristic Risk Calculation

We implemented a weighted formula to assign a numerical "threat score" to every device:

$$\text{Behavior Score} = (\text{Outgoing Connections} \times 5) + (\text{Incoming Connections} \times 3) + (\text{Infection History} \times 8)$$

This logic allows the system to predict attacks by analysing the structural properties of the graph (node degree) combined with historical state data.

3. Complexity Analysis (Performance Evaluation)

Per the requirements of Deliverable 3, we have analyzed the time complexity of our new implementations.

1. Infection Simulation (BFS)

- Operation: Visits every vertex (Device) and traverses every edge (Connection).
- Time Complexity: $O(V + E)$

Where V is the number of devices and E is the number of connections.

- *Justification*: Each node is enqueued and dequeued exactly once.

2. Patient Zero Analysis (DFS Wrapper)

- Operation: The system iterates through the list of devices. If a device is infected, it triggers a DFS.
- Worst Case Complexity: $O(V \times (V + E))$
- *Justification*: In the worst-case scenario (where all nodes are infected), we run a full DFS traversal ($V+E$) for every single node (V).
- *Optimization*: This is acceptable for our constraint of 200 devices. For larger networks, we would optimize this by using Dynamic Programming (memorization) to cache reachability results.

3. Risk Ranking (Selection Sort)

- Operation: Iterates through the array to find the maximum element and swaps it.
- Time Complexity: $O(N^2)$

Where N is the number of devices.

- *Justification*: For $N=200$, $N^2 = 40,000$ operations, which is instantaneous on modern hardware. If N were to grow to 10,000+, we would switch to Merge Sort ($O(N \log N)$).
-

4. Testing & Output Verification

Test Case 1: Behaviour Score & Sorting

- Scenario: We added 5 devices. Device "PC-1" has many outgoing connections and has been infected previously. We ran the "Show Behaviour Scores" function to see if the sorting logic correctly places PC-1 at the top.
- Observation: The Selection Sort correctly ranked the devices based on the calculated percentage.

Test Case 2: Locating Patient Zero (DFS)

- Scenario: We created a chain $A \rightarrow B \rightarrow C$. All three are infected.
 - If the infection started at C, it cannot reach A.
 - If it started at A, it reaches B and C (Count = 3).
 - The algorithm should identify A as the source.
- Observation: The locateInfectionSourceDFS function correctly identified the root of the cluster.

Test Case 3: Neighbour Infection Check

- **Scenario:** A user wants to check the safety of a specific node. We utilized the `checkConnectedInfection` function.
-

5. Conclusion

In this phase, we successfully integrated **Graph Traversal (DFS)** and **Sorting Algorithms** to transform the project from a simple simulator into an analytical tool. The system can now not only simulate how a virus spreads but also analyse *why* it spreads and *who* started it. The performance analysis confirms that our choice of algorithms is efficient for the defined scope ($N=200$ devices). The final phase will focus on code cleanup, final documentation, and presentation preparation.

